

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 6
По дисциплине «Современные платформы программирования»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
03.04.2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Ход работы:

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`
`@pytest.fixture`

```
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item):
```

```
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
```

```
coupons = {"SAVE10": 10, "HALF": 50}

if coupon_code in coupons:
    cart.apply_discount(coupons[coupon_code])
else:
    raise ValueError("Invalid coupon")
```

- Напишите тесты на apply_coupon
- Замокните словарь coupons с помощью monkeypatch или patch.dict

shopping.py:

```
import requests
```

```
class Item:
    def __init__(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.name = name
        self.price = price
```

```
    def __repr__(self):
        return f"Item(name={self.name}, price={self.price})"
```

```
class Cart:
    def __init__(self):
        self.items = []
        self.discount_percent = 0
```

```
    def add_item(self, item):
        self.items.append(item)
```

```
    def total(self):
        subtotal = sum(item.price for item in self.items)
        return subtotal * (1 - self.discount_percent / 100)
```

```
    def apply_discount(self, percent):
        if percent < 0 or percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        self.discount_percent = percent
```

```
def __repr__(self):
    return f"Cart(items={len(self.items)}, discount={self.discount_percent}%)"
```

```
coupons = {"SAVE10": 10, "HALF": 50}
```

```
def apply_coupon(cart, coupon_code):
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

```
def log_purchase(item):
    url = "https://example.com/log"
    requests.post(url, json={"name": item.name, "price": item.price}, timeout=5)
```

test_cart.py:

```
from unittest.mock import patch
import pytest
from shopping import Item, Cart, apply_coupon, log_purchase
```

```
@pytest.fixture
def cart():
    return Cart()
```

```
class TestCart:
    def test_add_item(self, cart):
        test_cart = cart
        test_cart.add_item(Item("Apple", 10.0))
        assert len(test_cart.items) == 1
```

```
    def test_negative_price(self):
        with pytest.raises(ValueError):
            Item("Bad", -5.0)
```

```
    def test_total(self, cart):
        test_cart = cart
        test_cart.add_item(Item("Apple", 10.0))
        test_cart.add_item(Item("Banana", 20.0))
        assert test_cart.total() == 30.0
```

```
@pytest.mark.parametrize("discount,expected", [(0, 30.0), (50, 15.0), (100, 0.0)])
def test_apply_discount(self, cart, discount, expected):
    test_cart = cart
    test_cart.add_item(Item("Apple", 10.0))
    test_cart.add_item(Item("Banana", 20.0))
    test_cart.apply_discount(discount)
    assert test_cart.total() == expected
```

```
@pytest.mark.parametrize("discount", [-10, 110])
def test_invalid_discount(self, cart, discount):
    test_cart = cart
    with pytest.raises(ValueError):
        test_cart.apply_discount(discount)
```

```
class TestCoupon:
    def test_apply_save10(self, cart):
        test_cart = cart
        test_cart.add_item(Item("Apple", 100))
        apply_coupon(test_cart, "SAVE10")
        assert test_cart.discount_percent == 10
```

```
def test_apply_half(self, cart):
    test_cart = cart
    test_cart.add_item(Item("Apple", 100))
    apply_coupon(test_cart, "HALF")
    assert test_cart.discount_percent == 50
```

```
def test_invalid_coupon(self, cart):
    test_cart = cart
    with pytest.raises(ValueError):
        apply_coupon(test_cart, "INVALID")
```

```
def test_mock_coupons_with_patch_dict(self, cart):
    test_cart = cart
    test_cart.add_item(Item("Apple", 100))
    with patch.dict("shopping.coupons", {"TEST": 20}, clear=True):
        apply_coupon(test_cart, "TEST")
        assert test_cart.discount_percent == 20
```

```
class TestLogPurchase:
    @patch("shopping.requests.post")
    def test_log_purchase_calls_post(self, mock_post):
        item = Item("Apple", 10.0)
        log_purchase(item)
```

```
mock_post.assert_called_once_with("https://example.com/log", json={"name": "Apple",  
"price": 10.0}, timeout=5)
```

```
test_cart.py::TestLogPurchase::test_log_purchase_calls_post PASSED [100%]  
===== 13 passed in 1.22s =====
```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

lab1.py:

```
import random
```

```
def generate_random_sequence(n):  
    numbers = list(range(1, n + 1))  
    random.shuffle(numbers)  
    return numbers
```

```
def find_majority_element(nums):  
    for x in set(nums):  
        if nums.count(x) > len(nums) // 2:  
            return x  
    return None
```

```
if __name__ == "__main__":  
    print(generate_random_sequence(5))  
    print(find_majority_element([1, 2, 2, 2, 3]))
```

tests.py:

```
from lab1 import generate_random_sequence, find_majority_element
```

```
class TestGenerateRandomSequence:  
    def test_length(self):  
        result = generate_random_sequence(5)  
        assert len(result) == 5
```

```
def test_contains_all_numbers(self):
    result = generate_random_sequence(5)
    assert sorted(result) == [1, 2, 3, 4, 5]
```

```
def test_zero_n(self):
    result = generate_random_sequence(0)
    assert not result
```

```
def test_one_element(self):
    result = generate_random_sequence(1)
    assert result == [1]
```

```
class TestFindMajorityElement:
    def test_majority_exists(self):
        assert find_majority_element([3, 2, 3]) == 3
```

```
def test_majority_exists_long(self):
    assert find_majority_element([2, 2, 1, 1, 1, 2, 2]) == 2
```

```
def test_no_majority(self):
    assert find_majority_element([1, 2, 3, 4]) is None
```

```
def test_single_element(self):
    assert find_majority_element([1]) == 1
```

```
def test_two_elements_same(self):
    assert find_majority_element([1, 1]) == 1
```

```
def test_two_elements_different(self):
    assert find_majority_element([1, 2]) is None
```

```
def test_empty_list(self):
    assert find_majority_element([]) is None
```

```
def test_all_same(self):
    assert find_majority_element([5, 5, 5, 5, 5]) == 5
```

```
tests.py::TestFindMajorityElement::test_empty_list PASSED [ 91%]
tests.py::TestFindMajorityElement::test_all_same PASSED [100%]
===== 12 passed in 0.04s =====
```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

1) Реализуйте и протестируйте метод `String repeat(String pattern, int repeat)`, который строит строку из указанного паттерна, повторённого заданное количество раз.

string_repeat.py:

```
def repeat_string(input_str, separator, repeat_count):
    if input_str is None or separator is None:
        raise TypeError("Arguments cannot be None")

    if repeat_count < 0:
        raise ValueError("Repeat count cannot be negative")

    if repeat_count == 0:
        return ""

    str_clean = input_str.strip()

    if repeat_count == 1:
        return str_clean

    result = str_clean
    for _ in range(repeat_count - 1):
        result += separator + str_clean

    return result
```

test_string_repeat.py:

```
import pytest
from string_repeat import repeat_string

def test_zero_repeat():
    assert repeat_string("e", "|", 0) == ""

def test_three_repeat():
    assert repeat_string("e", "|", 3) == "e|e|e"
```



```
def test_with_spaces():
    assert repeat_string(" ABC ", ",", 2) == "ABC,ABC"
```

```
def test_empty_separator():
    assert repeat_string(" DBE ", "", 2) == "DBEDBE"
```

```
def test_one_repeat():
    assert repeat_string(" DBE ", ":", 1) == "DBE"
```

```
def test_negative_repeat():
    with pytest.raises(ValueError):
        repeat_string("e", "|", -2)
```

```
def test_empty_string():
    assert repeat_string("", ":", 3) == ":::"
```

```
def test_none_pattern():
    with pytest.raises(TypeError):
        repeat_string(None, "a", 1)
```

```
def test_none_separator():
    with pytest.raises(TypeError):
        repeat_string("a", None, 2)
```

```
test_string_repeat.py::test_empty_string PASSED [ 44%]
test_string_repeat.py::test_none_pattern PASSED [ 88%]
test_string_repeat.py::test_none_separator PASSED [100%]

===== 9 passed in 0.06s =====
```

Вывод работы: освоил приемы тестирования кода на примере использования пакета pytest